Project Report

Project Title: The Bloom of Death

Course: COMP-2800, Xiaobu Yuan

Team Members: Hanan Senah, Lynn Hajj Hassan, Noor Haddad, Marco Lee-Shi, Fahim Kabir.

Table of Contents

1. Introduction

1.1 Purpose of the Project

The primary objective of this project was to design and develop a small interactive game, realizing our passion for game development despite the challenges of working with an older technology (Java 3D, last updated in 2005). This project served as a platform to apply and reinforce the knowledge acquired and shared in Comp 2800. Complemented by additional research from various online resources, including technical documentation and educational videos.
Our goal was to create a functional and immersive gaming experience while leveraging industry-standard methodologies such as object-oriented programming (OOP), 3D rendering, and real-time interaction handling. By integrating these techniques, we aimed to produce a well-structured and engaging application that demonstrates both our technical skills and creative vision.

1.2 Issues to be Discussed and Their Significance

- Technical Complexity – Handling advanced 3D rendering, animations, and transformations.
- Optimization and Performance – Maintaining smooth operation while delivering high-quality visuals.
- User Interaction – Designing intuitive controls and responsive feedback for an engaging experience.
- Collaboration and Project Management – Effectively coordinating tasks within a Scrum-based workflow.

Each of these challenges was critical to address, as overcoming them ensured a functional, efficient, and immersive final product that met our project goals.

1.3 Project's Methods

1-Scrum Methodology:

We followed an agile approach, organizing the project into sprints with continuous iterations. This enabled efficient task management, adaptability, and iterative improvements based on feedback.

2-Object-Oriented Programming (OOP):

Our codebase follows OOP principles, ensuring modular, reusable, and maintainable components. This approach improves code organization, scalability, and ease of debugging.

3-JogAmp (JOGL - Java OpenGL):

We used JogAmp's JOGL to facilitate hardware-accelerated 3D rendering in Java. This library provided direct access to OpenGL, enabling complex graphics processing, real-time rendering, and smooth animations.

4-Java 3D:

Java 3D was utilized for scene graph management, enabling efficient 3D object rendering, transformations, and hierarchical structuring. It simplified complex 3D operations and interactions.

5-TransformGroups & Alpha Class:

TransformGroups allowed structured positioning and movement of 3D objects, while the Alpha Class handled time-based animations, such as oscillations and rotations, to create smooth motion effects.

6-Integrated Development Environment (IDE):

Development was carried out in Eclipse, providing debugging tools, version control support, and efficient project management.

7-Version Control (Git/GitHub):

We used Git and GitHub for version control, enabling collaborative development, tracking changes, and managing code repositories efficiently.

8-Blender:

We used Blender when the scene needed complicated assets. It offered a comprehensive method to export 3D models in compatible formats suitable for Java 3D rendering.

1.4 Limitations and Assumptions

Limitations

- Performance Constraints – Real-time 3D rendering, especially with animations and collision detection, can be demanding on system resources, potentially affecting performance on lower-end hardware.
- Basic Post-Processing Features – Effects such as adding blur to the camera view have proven to be too intricate to integrate efficiently into the final product.
- Graphics API Dependency – The project relies on JogAmp's JOGL and Java 3D, which may have compatibility issues with certain systems or require specific configurations to work optimally.
- Complex Collision Detection – While basic collision detection is implemented, more complex physical simulations (e.g., soft-body physics or advanced collision handling) are not supported.
- UI Simplicity – The user interface focuses on core functionality and interaction but may not offer advanced or highly polished graphical elements.
- Limited Model Importing – External 3D models need to be manually adjusted for compatibility, and the project does not automatically process complex models from external tools without additional setup.

Assumptions

- System Requirements – It is assumed that the user's system supports JOGL, Java 3D, and necessary OpenGL drivers, ensuring proper rendering and interaction.
- User Familiarity – Users are assumed to have basic knowledge of interacting with 3D environments, including navigating the interface and understanding basic collision mechanics.
- Consistent Development Environment – The project is assumed to be developed and tested primarily in Eclipse, ensuring stability across different machines.
- Collision Detection Accuracy – The assumption is made that the implemented collision detection is sufficient for the current scope of the project, without the need for highly detailed physics simulations.
- Hardware Compatibility – The project assumes the system has the required processing power and GPU compatibility to handle both UI interactions and 3D object collision detection without major performance issues.

2. Project Management with Scrum

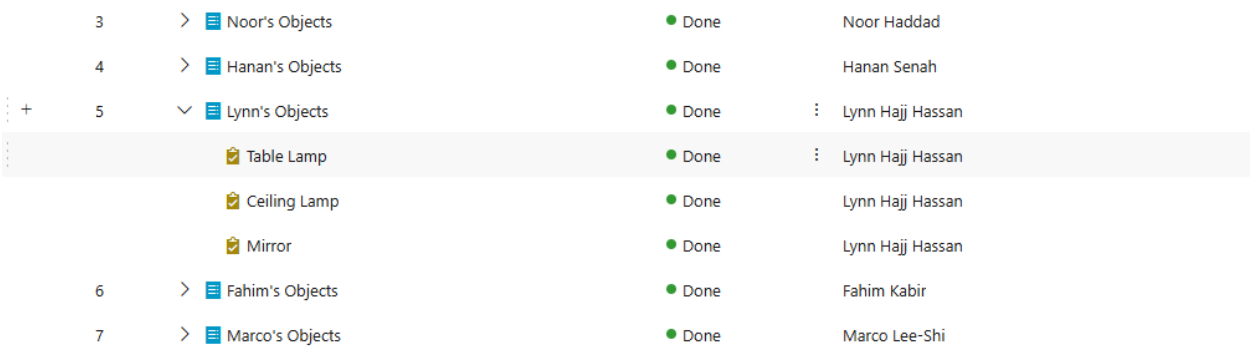
Throughout the project, we followed the Scrum methodology to organize our workflow and manage team collaboration effectively. Scrum helped us structure our tasks into sprints, assign responsibilities, and maintain continuous progress through backlog tracking and team coordination.

2.1 Use Cases as Backlog Items

To structure our work, we broke the entire project into **four main sprints**, each focusing on a specific development phase. Instead of traditional user stories, we treated **team members' contributions** as **backlog items**, with each backlog titled after the responsible team member (e.g., "Lynn's Objects", "Noor's Objects").

Under each backlog, we listed **specific tasks** that related to creating 3D assets for the game. For example:

- **Backlog:** Lynn's Objects
    - **Tasks:** Create the table lamp, create the ceiling lamp, create the mirror



This structure helped us clearly assign responsibilities while tracking progress for each member.

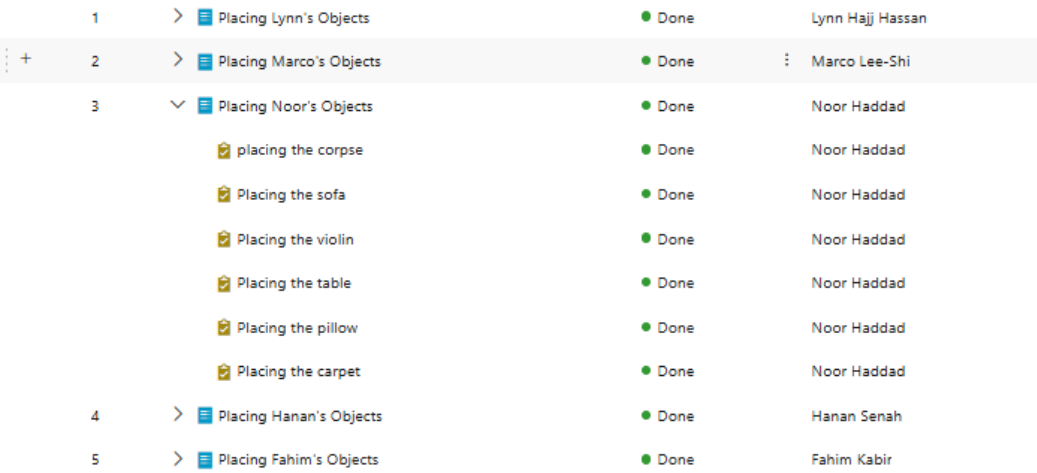
2.2 Creation of Tasks

Once the backlog items were defined, we held team meetings to **discuss and break them down into smaller actionable tasks**. These tasks represented tangible deliverables, such as modeling specific objects, placing them in the environment, or implementing interactions.

Tasks were written under the appropriate backlog item and were assigned based on each team member's area of work. For example:

- **Backlog:** Placing Noor's Objects

- **Tasks:** Place the corpse, place the table in the room, etc.



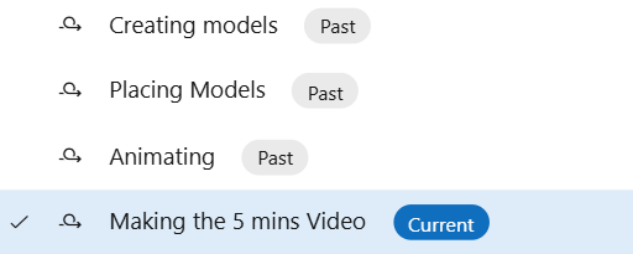
	1	Placing Lynn's Objects	Done	Lynn Hajj Hassan
+	2	Placing Marco's Objects	Done	Marco Lee-Shi
	3	Placing Noor's Objects	Done	Noor Haddad
		placing the corpse	Done	Noor Haddad
		Placing the sofa	Done	Noor Haddad
		Placing the violin	Done	Noor Haddad
		Placing the table	Done	Noor Haddad
		Placing the pillow	Done	Noor Haddad
		Placing the carpet	Done	Noor Haddad
	4	Placing Hanan's Objects	Done	Hanan Senah
	5	Placing Fahim's Objects	Done	Fahim Kabir

the Scrum Master (Lynn) , updated the Scrum board after every meeting to reflect the team's decisions and ensure that everyone was aligned.

2.3 Planning and Carrying Out the Sprint

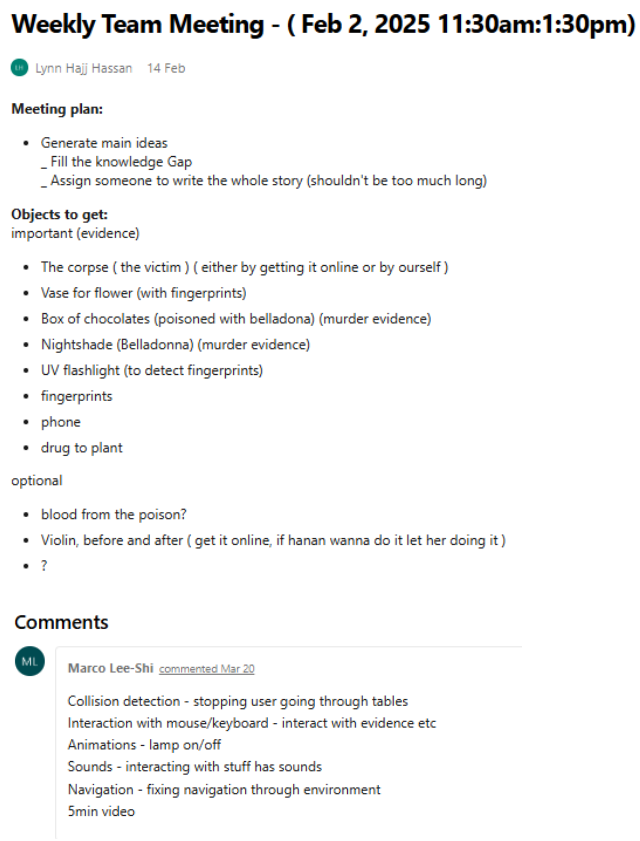
Our project consisted of **four sprints**:

1. **Sprint 1 (2 weeks)** – Creating 3D models
2. **Sprint 2 (1 week)** – Placing the models inside the 3D room
3. **Sprint 3 (2 weeks)** – Implementing user interaction, animation, and audio
4. **Sprint 4 (1 day)** – Producing the final project video



Before each sprint, we met to agree on what tasks needed to be completed and how they would be divided among team members. Tasks were tracked using a **Scrum board**.

And the meeting's information was tracked using **Scrum wiki**:



Weekly Team Meeting - (Feb 2, 2025 11:30am:1:30pm)

Lynn Hajj Hassan 14 Feb

Meeting plan:

- Generate main ideas
  _ Fill the knowledge Gap
  _ Assign someone to write the whole story (shouldn't be too much long)

Objects to get:
important (evidence)

- The corpse (the victim) (either by getting it online or by ourself)
- Vase for flower (with fingerprints)
- Box of chocolates (poisoned with belladona) (murder evidence)
- Nightshade (Belladonna) (murder evidence)
- UV flashlight (to detect fingerprints)
- fingerprints
- phone
- drug to plant

optional

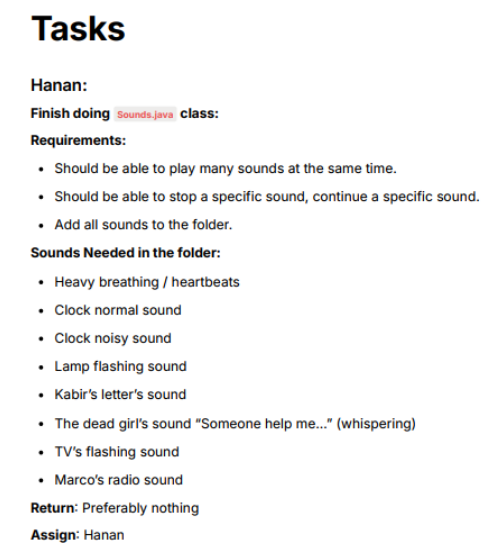
- blood from the poison?
- Violin, before and after (get it online, if hanan wanna do it let her doing it)
- ?

Comments

ML **Marco Lee-Shi** commented Mar 20

Collision detection - stopping user going through tables
Interaction with mouse/keyboard - interact with evidence etc
Animations - lamp on/off
Sounds - interacting with stuff has sounds
Navigation - fixing navigation through environment
5min video

We also used **Notion** for writing detailed requirements and GitHub to share files.



Tasks

Hanan:

Finish doing Sounds.java **class:**

Requirements:

- Should be able to play many sounds at the same time.
- Should be able to stop a specific sound, continue a specific sound.
- Add all sounds to the folder.

Sounds Needed in the folder:

- Heavy breathing / heartbeats
- Clock normal sound
- Clock noisy sound
- Lamp flashing sound
- Kabir's letter's sound
- The dead girl's sound "Someone help me..." (whispering)
- TV's flashing sound
- Marco's radio sound

Return: Preferably nothing

Assign: Hanan

While we faced challenges early on—particularly with understanding how to use Scrum—we gradually adapted and became more comfortable with the methodology as the project progressed.

2.3 Team Management

Our team consisted of **five members**:

Lynn Hajj Hassan: Responsible for maintaining the Scrum board, documenting sprint plans, ensuring task distribution, and contributing as a developer.

Marco Lee-Shi: Assisted in recording progress, organizing missing pieces, and providing ideas to team members. Also worked as a developer.

Hanan Senah: Responsible for tracking deadlines and worked as a developer.

Fahim Kabir & Noor Haddad: Developers.

We managed communication through:

- **Online meetings** for urgent issues
- **In-person group work sessions** for coding together
- **GitHub** for collaboration and file sharing
- **Notion** for requirement documentation

We minimized conflicts by carefully planning and writing clear, assigned tasks. Each member worked on **separate classes**, and we agreed to **integrate code at the end**, avoiding overlapping or merge issues.

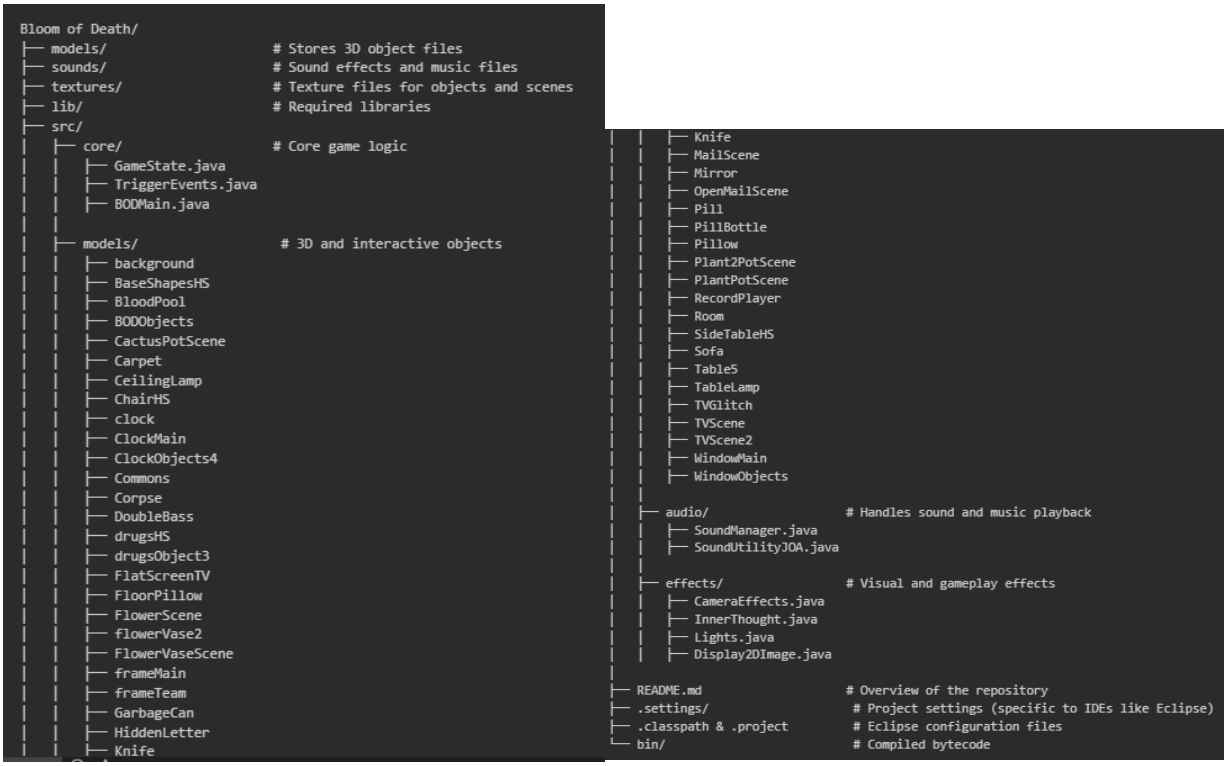
To stay on track, we regularly checked in on each member's progress and occasionally held **one-on-one meetings** to resolve blockers and provide support.

3. (Object-Oriented) Software Development

3.1 Overall Project's Implementation

In developing our project, we adopted object-oriented principles for better structure,and maintainability. The codebase was modularized into well-organized packages, with each class focusing on a single responsibility. Below are the key aspects of our software development process.

- **Project's Structure :**



Inside the src/ directory, we divided the codebase into multiple packages, each serving a distinct purpose:

_audio/ package: Managed all audio-related functionalities.

effects/ package: Contained classes responsible for visual and environmental effects in the game.

core/ package: Central to the game's logic and state management.

models/ package: Included all 3D model classes used in the game room environment. Each model was encapsulated in its own class, such as TableLamp, Corpse, Room, ClockMain, etc.

This structure allowed us to isolate components, making the code easier to navigate, maintain, and test.

Additionally, many classes followed inheritance patterns—for example, most 3D model classes extended a base class like BODObjects.

- **Techniques of Implementation**

The project was implemented in **Java** using **Eclipse IDE**. We followed object-oriented design principles, notably:

_Encapsulation:

Each model was implemented in its own class, with a dedicated method for creation.

Example: Mirror.create_mirror() was a static method used to create and return the model's transform group.

_Modularity:

Code was divided into logical packages (audio, core, models, effects) and supported by separate folders for textures, sounds, and model files (.obj files).

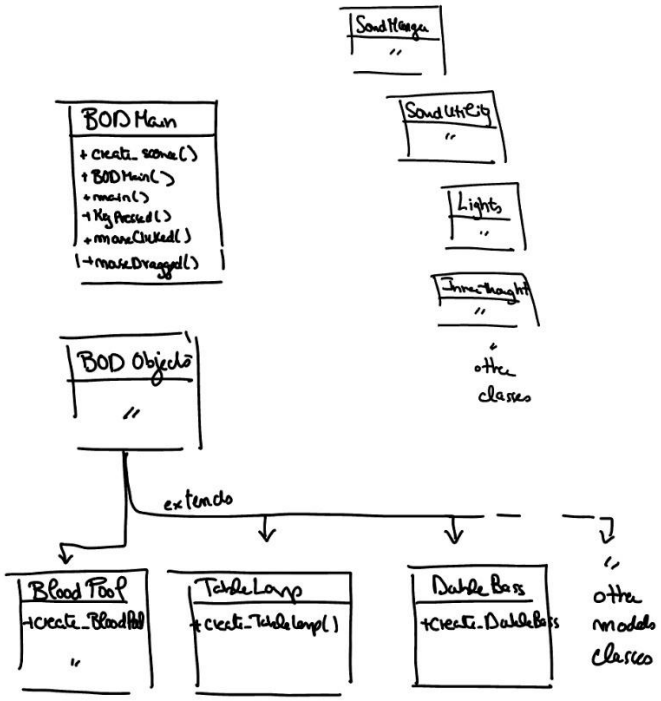
_Inheritance:

Used in multiple areas such as input handling (e.g., mouse and key listeners in BODMain) and model hierarchies where many objects inherited from a base class (BODObjects).

_Reusability:

The structure made it easy to reuse components and call specific object creation methods in the main logic (BODMain).

Assets like sounds and models were neatly stored in respective folders, keeping the project organized and scalable.



- **Software Testing and Operation**

Testing was mostly done manually but was effective due to the modular design of our classes. Since each component was built separately, we were able to:

_Test individual model classes in isolation by calling their creation methods and observing behavior in the scene.

_Debug using Eclipse's built-in debugger, allowing us to trace logic and resolve bugs in real-time.

_Quickly identify and fix issues thanks to the clean separation between packages and classes

3.2 Lynn's Part

- **Identification of classes**

_GameState: The GameState class is responsible for tracking the clues that the player has found throughout the game. It holds a set of clues and provides methods to check if a clue has been found and to add new clues as the game progresses.

_ TriggerEvents: This class handles the triggering of specific events based on the player's interactions with objects in the game world. The triggerEvent method displays messages, plays sounds, and manipulates visual effects, like shaking the camera or making lights flicker.

_BODMain: The BODMain class serves as the entry point of the game and orchestrates the creation of the game world. It handles the initialization of the room, models (like lamps and mirrors), and the interaction between the player and objects. It also ties together the GameState and TriggerEvents classes to ensure that the game progresses when clues are found or events are triggered.

_BODObjects: This is a base class for objects in the game. Other model classes like CeilingLamp, Mirror, Room, and TableLamp extend BODObjects. These classes handle the creation and manipulation of objects, such as loading textures, applying transformations, and controlling object attributes.

_CeilingLamp, Mirror, TableLamp, Room: These classes represent specific objects in the game environment. Each one is responsible for creating its respective object (e.g., lamp, mirror) and handling the 3D transformation (positioning, rotation) of the object in the room.

_Lights: The Lights class controls the lighting in the game environment, including ambient lighting, Directional light, exponential fog, etc. It enhances the atmosphere of the room.

_InnerThought: This class is used to display inner thoughts as text bubbles when the player interacts with specific objects. It provides visual feedback about the player's actions or thoughts, contributing to the narrative.

- **Software Design**

The relationships between these classes are centered around composition and inheritance:

_BODMain manages the game world and integrates other classes like GameState, TriggerEvents, and Lights. It is the central hub for game logic and state management.

_BODObjects is the parent class for specific object models like CeilingLamp, Mirror, and Room. These subclasses handle the creation and manipulation of 3D models.

_TriggerEvents uses InnerThought to display messages and SoundManager for playing sounds based on events. It also interacts with GameState to update the player's progress and track clues.

- **Techniques of implementation**

_Modular Design:

Implemented modular classes, like BODObjects and its subclasses (CeilingLamp, Mirror), to make it easier to manage and extend game objects. This approach allows you to add new objects or change existing ones without affecting the core game logic.

_Event-Driven Programming:

The TriggerEvents class handles events based on player interactions. When a specific object is interacted with, a corresponding event is triggered, such as displaying a thought, playing a sound, or applying visual effects like shaking the camera.

_3D Rendering with Java 3D:

Used Java 3D's TransformGroup to manage the positioning and transformations of objects in the game world. Each object class, like CeilingLamp, provides a method (e.g., create_CeilingLamp) to create its 3D representation, which is then added to the roomTG in BODMain.

- **Software testing and operation**

Unit Testing: I manually tested each method using a debugger to ensure that everything works as expected. This involved testing the creation of objects, the handling of interactions, and the correct triggering of events.

Debugging: I used the debugger to track how models are being created and attached to the scene graph (e.g., how TransformGroup objects like CeilingLamp are added to roomTG).

Interaction Testing: Testing how events are triggered when interacting with objects, such as finding clues or interacting with objects like the "mail

Visual Feedback: I also tested the visual feedback, like the display of inner thoughts and the application of lighting effects, ensuring they appear at the right times.

3.3 Marco's Part

Identification of Classes:

- BloodPool.java
- Display2DImage.java
- FloorPillow.java
- GarbageCan.java
- Knife.java
- RecordPlayer.java

To implement my 3D objects, I followed a consistent object-oriented structure using a shared base class, BODObjects. This base class provided common methods for applying transformations, positioning, and setting up appearances and textures. For each object (e.g., BloodPool, FloorPillow, Knife, GarbageCan, RecordPlayer), I extended this base and followed a clear pattern: set the scale and position, load the 3D model using transform_Object(), define material colors, and apply a texture using TextureAttributes and scaled Transform3D.

When creating objects composed of multiple parts—like the Knife, which consists of a handle and blade—I used an array of BODObjects to manage each component. Each part was created as a separate subclass (e.g., Handle, Blade) with its own appearance and position. I then attached the components together hierarchically using add_Child() and grouped them under a main TransformGroup, allowing for unified transformations such as rotations. This modular approach made it easy to construct and manage more complex objects while keeping the code clean and reusable.

The RecordPlayer class is a fully modeled and animated composite 3D object composed of nine individual parts, each represented by its own class that extends from a shared base class, BODObjects. These components include the RecordBase, Pivot, Platter, Rarm, Rweight, Rspeed, Rcut, Rcue, and the animated Record itself. Each part is responsible for setting its own scale, position (post), and textured appearance through the create_Appearance() method, following the structured object template I developed. I used a BODObjects[] array to manage all the components, which allowed me to instantiate, access, and connect them in a clean and ordered way.

The hierarchy of the player is built using TransformGroups, with each part being attached to its parent using the add_Child() method. This hierarchical setup allows for relative positioning—for example, the Rarm is connected to the Pivot, and the Rweight is then attached to the Rarm—maintaining proper structure and movement flow. A global TransformGroup is used to apply a 90-degree Y-axis rotation to the entire record player so that it aligns properly within the 3D scene.

A key feature of this class is the animated Record, which is made to continuously spin using a RotationInterpolator controlled by a looping Alpha object. The transformation is applied via a separate rotation group (objRG) within the record's TransformGroup, and a scheduling bounds (BoundingSphere) ensures the animation remains active within the visible area. Each part also includes texture attributes and scaling via Transform3D, allowing precise control over how textures like matte black, silver, and red-button appear across different surfaces.

One of the most challenging parts of creating the record player was manually positioning each object by hand. Since the parts were modeled separately and loaded into the scene

programmatically, I had to iteratively adjust their position values, run the program, and observe how each part rendered in the 3D space. Fine-tuning the exact coordinates, especially for smaller components like the buttons or the pivot arm, required a lot of trial and error. There were no visual layout tools involved—just observation, tweaking values, and reloading—making it a time-consuming but rewarding process that helped me better understand spatial relationships in Java 3D.

This modular, array-based, and transformation-driven design not only keeps the code organized but also makes it highly reusable and scalable for more complex assemblies. It showcases how multiple 3D objects can be composed into a single functioning unit with animation and structure, while still following a consistent development pattern across the project.

To support temporary on-screen effects and UI feedback, I implemented a Display2DImage class using Java Swing. This class allows me to display 2D images—such as icons, blood splatters, pop-ups, or hints—on top of the 3D scene for a short duration. This is useful for adding immersive visual cues or game feedback without affecting the 3D rendering pipeline. The class works by accessing the JLayeredPane of the main game JFrame, which allows for layering Swing components above the canvas that renders Java3D. When I call showImage(), it takes parameters for the image path, position (x, y), size (width, height), and display duration in milliseconds.

Detailed testing approaches & Summarize test results and bug fixes

To ensure integration and reliability across the project, I performed manual testing after each implementation phase. Once the objects and systems were in place, I ran the full code/game to verify that all 3D components were rendering correctly, positioned as expected, and behaving as intended. In addition to local testing, I also shared the codebase with another group member to run it on a different machine, which helped verify compatibility and ensured that there were no environment-specific issues.

For performance testing, I monitored how long the game took to load and render all components during startup. To improve efficiency and minimize loading times, I deliberately selected low file size textures without sacrificing visual quality. This helped speed up texture loading and reduced memory usage, especially important when dealing with multiple textured models in a complex 3D scene.

During development, I encountered a few issues. A notable one involved the Display2DImage class—initially, 2D images were not being displayed as expected. After debugging, I discovered that the issue stemmed from a conflict between the JFrame used for the game and an unintended additional frame or incorrect use of layers. The image overlay was being added to the wrong

frame or layered pane, which prevented it from rendering visibly. This was resolved by ensuring the correct reference to the game's primary JFrame and using the proper JLayeredPane for overlays.

After addressing these bugs and optimizing performance, the application now runs smoothly, with all components rendering correctly and animations functioning as intended across multiple test environments.

3.4 Hanan's Part

Identification of classes

The key classes that I implemented and used in the project are:

1. **ChairHS**.java
2. **ClockMain**.java - **ClockObject4**.java
3. **DrugsHS**.java - **drugsObject3**.java
4. **WindowMian**.java -**WindowObjects**.java
5. **Background**.java
6. **SideTableHS**.java
7. **SoundManager**.java
8. **FrameMain**.java - **frameTeam**.java

My Key Class and its implementation

The classes I implemented are used in the main class of the project, BODMain.java. One of these classes is ChairHS.java, which I created using Java3D. It is called in BODMain.java to place a 3D chair model next to the large table. This class is responsible for creating and managing the 3D chair within the application. Inside ChairHS.java, there is a nested class ChairHS2, where I used 3D primitive shapes such as Box, Cylinder, and Sphere from the org.jogamp.java3d.utils.geometry package. I used TransformGroup to combine these shapes, and applied multiple transformations—such as Transform3D, setTranslation, rotY, and setScale—to achieve the desired layout of the chair components.

Similarly, the SideTableHS.java class was fully created using Java3D. It is called in BODMain.java and placed next to the sofa, serving as the table where the lamp is positioned, with one of the flowers placed beneath it between the legs. This class constructs a 3D side table model using Java3D primitives such as Cylinder for the tabletop, legs, and shelf. Various transformations—such as translation, rotation, and scaling—are applied to accurately position and shape these

components. A custom method, createWickerAppearance(), is used to load a texture and apply material properties to the objects, giving the table a realistic appearance. The table legs are arranged using mathematical techniques to ensure even spacing. The class also sets up a simple camera view and displays the scene in a JFrame using SimpleUniverse.

In DrugsHS.java and drugsObject3.java, I created a 3D model of a pill bottle with scattered pills using Java 3D. The model is placed next to the corpse in BODMain.java to create ambiguity about the cause of death. The bottle and cap were built using cylinders, while pills were formed with a cylinder and two spheres. Materials and textures were applied for realism, using red and yellow for the pill ends. Pills were randomly scattered around the bottle using polar coordinates and rotated with rotZ to lie flat. drugsObject3.java includes a main method to display the model, following lab and assignment structure.

The classes Clockmain.java and ClockObject4.java are both used in BODMain.java to place the clock object in the room. Similar to Assignments 3 and 4, I followed an object-oriented design approach by creating several concrete subclasses that represent different parts of the clock. These parts—such as the house, base, center, top, sides, windows, and trees—are each modeled as separate classes.

The abstract class ClockObject4 defines the general structure for clock components. It provides methods for positioning, transformation, and setting appearances, manages external object loading and transformations, and includes basic sound functionality (though not fully implemented). I later created a dedicated sound class that could be used by all group members, which I will explain separately.

To create the subclasses, I first searched online for a 3D model of a clock and downloaded it as an .obj file. I then imported the file into Blender. Initially, I attempted to merge two textures into one, which took about two hours. However, I soon realized that merging the textures caused the clock parts I intended to animate to behave as a single object, which wasn't suitable. To fix this, I re-imported the .obj file into Blender, selectively hid the parts I didn't need, and exported each component separately. I ended up with 13 individual .obj files, which I organized into a folder named Clockimages. These files were then used to build the complete clock in the application.

As a result, the **subclasses** are: **ClockHouse** - Represents the main structure of the clock. **ClockBase** - The base of the clock. **ClockCenter** - The center of the clock (contains a circular component This is an example of composition, where an additional 3D shape (Cylinder) is created and attached to the ClockCenter.). **ClockTop** - The top section of the clock. **ClockSide1**, **ClockSide2** - The sides of the clock. **ClockWindowobj** - A window object on the clock. **ClockTree1**, **ClockTree2** - Trees in the scene. **TreeBase1**, **TreeBase2**- Base of the trees.

Each one of them I applied to them a transformations and Positioning like each object is given a specific position (Vector3f) in the 3D space: post = new Vector3f(0.02f, 0.77f, -0.1f); used this technique to ensure correct placement relative to other objects in the scene. Scaling (scale) - Controls size. Rotation (rotY()) - Rotates around the Y-axis. Translation (setTranslation()) - Moves the object. Each object is added to a parent-child hierarchy using TransformGroup. Textures are applied to objects using obj_Appearance(). So, it loads an image texture onto the 3D shape and uses Java 3D's Material & TextureLoader for rendering. The subclasses that used animation and transformation to them, are:

HourHand Animation

Purpose: The hour hand rotates smoothly to represent the passage of 12 hours.

- **Scaling & Positioning:** The hour hand is scaled to 0.4d and positioned slightly above the motor. Initially rotated flat on the XY plane (rotX(Math.PI / 2)).
- **Rotation Animation:** Uses a RotationInterpolator to animate the rotation. The hand completes a full 360-degree rotation in 21600 seconds (6 hours) (the time in the game is faster for two purposes, one for the game thema that the time f=got effected by the crime and when the girl died the time got faster, psychological part, second, I asked the professor and he told me that I need to show the animation faster for the clock as the video is only 5 minutes, and the class will not stay entire hour to see it). The transformation is set up to maintain a clockwise rotation. Alpha(-1, 21600 * 1000) ensures indefinite looping of animation.
- **Manual Rotation Update:** A function updateRotation(float angle) allows dynamic adjustment of the hand's rotation manually.

MinuteHand Animation

Purpose: The minute hand rotates every 60 seconds to represent the passage of time.

- **Scaling & Positioning:** The minute hand is slightly larger (0.45d) than the hour hand. Positioned similarly above the motor. Loaded using transform_Object("HourHand") and colored red.
- **Rotation Animation:** Uses RotationInterpolator to complete one full rotation every 60 seconds. Rotates clockwise with an interpolator ranging from 0.0f to -2.0f * Math.PI. The scheduling bounds (CommonsHS.twenty_BS) ensure the animation runs smoothly.
- **Manual Rotation Update:** Similar to the hour hand, updateRotation(float angle) allows manual positioning.

ClockDong Animation

Purpose: The pendulum swings back and forth to simulate a traditional clock dong.

- **Scaling & Positioning:** The pendulum is scaled at 0.7d and positioned beneath the clock. Adjustments are made to ensure the pivot aligns with the top of the stick.
- **Swinging Motion:** Uses a RotationInterpolator to oscillate between -10° and 30°. Alpha(-1, swingDuration / 2) ensures a smooth back-and-forth swing. The interpolation is defined to enable both increasing and decreasing phases, simulating a natural swing.
- **Rotation Axis:** The pendulum swings around the **X-axis** (rotX(Math.PI / 2)). Ensures a visually accurate swinging motion.

For the class frameTeam and the frameMain are used to create the frame object, in the same way I created the clock.

Similarly, WindowMain.java and WindowObjects.java are used to construct the window frame, following the same design and construction approach as the clock and frame.

The Background.java class is responsible for building the 3D background environment seen through the window, using Java 3D. I used this class to create the outside view visible through the window. It defines the dimensions of the room and includes methods for creating and positioning windows, applying the appropriate textures and transformations.

The implementation uses TransformGroup and Transform3D to apply scaling, rotation, and translation, ensuring that objects are placed accurately within the scene. Textures are mapped onto surfaces using TextureLoader, while material properties are defined to control the appearance of walls and windows. The class follows a structured approach, with separate methods for transformations, texture handling, and object creation, which enhances modularity and maintainability in the design.

The SoundManager class is designed to manage sound playback efficiently using the Singleton pattern, ensuring that only one instance of the class exists throughout the application. It achieves this by defining a private static instance and providing a public static method (getInstance()) to access it. The class uses a HashMap (loadedSounds) to track loaded sounds, preventing redundant loading and optimizing performance. Additionally, it employs composition by integrating an instance of SoundUtilityJOAL to handle sound operations such as loading, playing, stopping, pausing, and resuming sounds. To enable time-based sound playback, the class utilizes Java's Timer and TimerTask, allowing scheduled stopping of sounds after a specified duration through the playSoundForDuration method. The encapsulation of sound-related methods ensures modularity and maintainability, while the cleanup method helps manage resources by stopping

scheduled tasks and releasing OpenAL resources. Overall, the SoundManager class effectively centralizes sound control, making it reusable and efficient within the application.

Detailed testing approaches & Summarize test results and bug fixes.

While testing ClockDong on the submission night, I noticed the pendulum's swing was off due to a misplaced pivot. The issue was caused by an extra offset, which I fixed by dynamically adjusting the extraFix value based on the object's height. This made the swing appear smooth and natural.

In the room scene, the clock wasn't visible from afar initially. The problem was traced to the bounding sphere in Commons.java: public final static BoundingSphere twenty_BS = new BoundingSphere(new Point3d(), 20.0); Increasing the radius to 200.0 fixed the visibility issue completely.

For SoundManager, I used JUnit and Mockito for unit and integration testing, ensuring methods like loadSound(), playSound(), and resumeSound() worked correctly. Performance tests confirmed reliable multi-sound playback and effective resource cleanup.

I also fine-tuned Vector3f values for precise object placement, ensuring accurate transformations and seamless interactions in the scene.

3.5 Noor's Part

List and describe key classes used:

The key classes I have fully implemented overall are:

1. Carpet.java
2. Corpse.java
3. DoubleBass.java
4. Sofa.java
5. Pillow.java
6. Table5.java
7. Collidable.java
8. CameraEffects.java

The carpet is built entirely using Java3D code. Like my other 3D model classes, it encapsulates geometry, appearance, and position. The CarpetMain class handles the construction of the overall shape of the carpet. It applies scaling and positioning while defining the geometry using a QuadArray and includes texture coordinates for its surface. In addition, like most of my classes containing 3D models, it has collision detection capabilities in a bounding box.

The corpse is a novelty of a creation that took me 20 hours to complete fully. It is a 3D model taken from Blender, posed in the Blender 3D software and her skeleton proved unexpectedly complex, taking about 5 hours to fix the model's broken bones. For instance, moving the corpse's hands would move her entire hair model, and other different kinds of problems were faced in constructing her. I also baked her 9+ textures into one for Java3D compatibility.

DoubleBass.java is a 3D model of a double bass instrument that is accurately positioned relative to its real-life scale. The exact scaling was positioned using a TransformGroup. Like most classes, it encapsulates geometry, materials, and collisions using a bounding box, and is also pickable and interactive.

Sofa.java is a 3D model of a real-life-sized sofa that was partly made from scratch using Blender. I made the sides, and it took around 6 hours to fully model the sides for the sofa. Translation and Y-axis rotation were applied to position the object in its specified spot according to the general plan.

The pillow class came with the sofa, but I separated it due to differing textures and material fixes.

Table5 class has a similar implementation to the rest of the classes with models.

Collidable class is an integral part of implementing the collision detection system, allowing player-object interaction boundaries by efficient boundary box logic. I store the box's dimensions and exchange data of every object and control them in the collisionObstacles list for tracking

collision. I implemented the method isCollidingWithObstacles models the player as a bounding sphere and checks for intersection with any object's axis-aligned bounding box. Vector math was used to clamp the future position of the sphere to the closest point on every model's box. It calculates the squared distance to efficiently detect collisions. While shifting, the next position is calculated and invokes the method to ensure the player continues blocking shifts when detecting collisions. I have used print statements to display the x, y, and z positions for debugging where the player is colliding with.

The CameraEffects class contributes to the project by providing post-processing visual effects for the first-person view of the player. The camera fades to a red overlay and shakes heavily as if there is an earthquake. It attaches a transparent red box in front of the player's camera view dynamically. Simulating visual distortion and using a TimerTask to gradually fade the colour over time.

Explain coding methodologies and best practices:

In the initial phase of my code, it was one file named TheBloomofDeath.java that held everything in place including the room, lights, objects, and any other functionality. After noticing one of the group members splitting the main class into multiple classes, I figured I should make my methods into classes to introduce modularity. The code turned from methods for each 3D model to each model having its class. This has proven to be helpful because once each class returns their transformation group, I could simply spawn any model I want by simply attaching it to the room. Thus, demonstrating polymorphism that acts as an efficient way to work with tens of objects with ease. An excellent demonstration of good practice in code was my textured_App method located in the BODObjects.java class. It provides a quick and easy way to identify texture paths with debugging messages aiding in the search for any texture that failed to load. It dynamically tries to load a texture and that allows flexibility in supporting multiple image formats instead of only .jpg formats. Additionally, modularity has been maintained here since I am encapsulating the texture mechanics in its method which makes the process reusable across all models linked to the main class. The method also avoids hardcoding texture paths or assuming certain formats exist by scanning the right file formats and paths dynamically.

Now, shifting our focus to a feature of utmost importance, collision. isCollidingWithObstacles function serves as a collision detection method that uses an axis-aligned bounding box to make it clear where the player will be blocked from transitioning. It recognizes the player as a shifting sphere with a radius and checks if that sphere intersects with any objects in the scene. Through calculating the distance of the sphere (the player) from every object's

bounding box and using its radius to determine if a collision has occurred. When a collision event triggers, the method with lightning speed makes an effective check to deter the player from moving through the collision set next to it. I stored all collidable 3D models in a collisionObstacles list to easily manage collisions. Much playtesting and debugging of the code was needed to make collisions work. In later phases of the code, I made it possible to place certain collision boxes without relying on the 3D model to make its connections possible because some objects I made for my team members like the extra wall were not a similar class to my objects. Consequently, that led me to try many ways of generating a bounding box inside the main file with any geometry not tied to a .obj-based class. As a result, the method handles each model's custom dimensions: width, height, and depth to ensure the actual dimensions visually.

Techniques of implementation:

The strength of my classes came from two primary sources: fully working collisions and the high-quality models I used. I have explained how collisions work and why they matter in the previous section, so this section will cover the entire process of turning the 3D models taken from websites into Java3D objects. I first worked on getting the corpse model. I looked at specific models that were possible to integrate into a crime-themed scene. The model I ended up choosing to fit what I was looking for which had a skeleton I could pose, was realistic enough, and the appearance looked consistent with other objects in the scene. For instance, the aesthetics of both the corpse and sofa fit the scene. Early issues in posing the model were encountered when the corpse model would react incorrectly when shifting non-relevant parts of her body. For example, her hair fell off from her head entirely when raising her hand. I had to follow many tutorials to fix that issue. After spending a staggering 5 hours posing the corpse the next issue came up. I exported her model as .obj and when coding her model, I discovered that she has almost more than 9 different textures for each body part including her clothes. This led me to spend the next 5 hours working on separating every piece of her body into distinct sections for the arms, legs, head, and clothes meshes. After I was done with that, I discovered an even worse issue. I could not line up her parts correctly which showed me that my effort was fruitless. I came up with a final plan of combining all her textures into one texture. This is a known process in Blender called baking. The most recent version of Blender had texture baking issues which resulted in the textures rendering completely black. So, in each of my five tries, I failed at making a fully textured model from one texture. I had to utilize my Photoshopping skills using paint.net software to combine each failed try and construct one full texture that looked perfectly identical to the 9 textures. Through photoshop, I was able to dress the corpse in a more fitting and logical sense.

The sofa is another 3D model I had an issue with. The main problem is that I was tasked by the leader of the group to gather references for sofa models and choose the most fitting one for the plan. I scoured the internet to find a game-ready, plan-fitting model. However, I only found the middle piece of the sofa, so I had to get creative. In terms of creating 3D objects from scratch, I do not have the serious skills needed to be a professional 3D modeler, so I had to come up with something. The sides of the sofa were the problematic part I needed to construct from scratch. Many experiments were performed to get the shape right since the sofa sides are circular, and I needed a clean topology to subdivide the model when applying a subdivision modifier. Four total iterations were made until I got the loop cuts correct apart from flipped normals that were easy to remedy. Using my sculpting skills, I have used the cloth brush to move around the shape of the sides to make its appearance as close to the reference as possible. After 5 hours of focused work, I ended up with a mesh I was satisfied with.

Now comes putting all objects together. It had its fun challenges along the way. I made a couple of adjustments to movement for easier debugging to the BODMain class including making crouching possible to check if there are any clear model collisions. This has helped me and my team greatly because initially from a top-view angle, we could not see if the 3D objects were floating or not. It turned out that most of the objects I placed were floating in the air as well as my team members and using crouching greatly helped us pinpoint the correct y-axis suitable to be on top of the floor. The next feature that helped give a general idea of whether the objects were placed in their correct spots was the minimap. Unfortunately, we ended up not using the mini-map, and implementing it was not easy since I had to figure out how to make another view and display it as its window in one of the corners. I settled with this implementation where I have another universe separate from the first-person universe the player will be controlling. The pro of this approach is that I had a fully working mini-map to show me the view far away. The con was that I had to spawn the same objects twice because the mini-map universe was separated from the main universe. I could not find a way to attach both without creating syntax issues. When I figured out the correct view coordinates everything worked as intended but because the duplicated models increased runtime significantly, the team decided not to include it in the final implementation. However, it did indeed help us point out where every object should be.

My most helpful debugging tool ensured the models matched their Blender versions. To make sure there are consistent visuals in my Blender models and their Java3D counterparts, I implemented a system to adjust Java3D materials in real time. The system has allowed me to dynamically change and preview material properties like specular, ambient, diffuse, and emissive materials, as well as the shininess part of every material for objects. These values heavily impact how the model will appear because they control how the model emits and reflects light within the Java3D scene. Two essential methods make this system possible: registerForTweaking() and

updateAppearance(). Invoking registerForTweaking() links the object to a global callback (BODMain.updateCallback), which when triggered, calls updateAppearance(). In turn, this method invokes the reusable updateAppearance() function from BODObjects, where a material gets constructed, if it does not exist. Using setEmissiveColor(), setDiffuseColor(), setAmbientColor(), and setShininess(), the values get updated. Then, the material gets attached to the Appearance of the object and applied to the Shape3D using setAppearance(). Every essential capability of Java3D is turned on through setCapability(ALLOW_COMPONENT_READ) and setCapability(ALLOW_COMPONENT_WRITE), making sure real-time adjustments for materials are possible. Here is an example class that uses the system, the sofa class. Using registerForTweaking(), lets me preview how incrementing and decrementing different values could alter the overall look dramatically in the Java3D scene. Each material is controlled by a button and pressing the buttons would increase whichever value is tied to the button. If I wanted to decrease the value, I have an additional button that would act as a switch between increment and decrement mode. Also, pressing each button would report to me what value that material is currently at, so I continue playing with the values of each material. Once I am happy with the values, I can simply document each material's value and simply manually hardcode in these values inside the create_appearance() method so that every time the models would look at their absolute best. This workflow has given me the chance to correct each model's aesthetic to match what the render for the same model in Blender precisely and efficiently. The significance of the system is huge to me because before developing it, I had to sit around waiting for the program to run each time I made a change to the materials, so it saved me many minutes of waiting, which led me to achieve a consistent scene-wide look.

Software testing and operation:

The most effective way to test any code I wrote was to run the program and debug the feature by testing a few things. The process decomposes into two questions I ask myself when I click the run button. The first question is, "Will the feature I implemented work for some cases?" This question helps me check if the new code is at least partially working. It tells me that I am close to the final solution to make it work every time I run the program. I test it by playing around and trying to break it to the best of my ability. Assuming the feature works most of the time, I try to isolate the working factors and test any case that would break it completely. Then, I ask for feedback to my team members if they could solve the issue or I try to solve it myself. Once 9/10 cases the feature works, I move on to the next important thing to implement. I have used this technique on the material adjustment system I mentioned in the previous section.

The second question I ask myself is, “will any feature break due to my latest implementation?” This is an important one. Although the leader has done a great job at making the entire code structure as modular as possible, it still does not guarantee that there will not be any issues with a new system being added. A good example of that question appeared when one of my team members had implemented the thought system. Her system unintentionally interfered with the collision logic I had spent significant time perfecting. The solution was effective communication between both of us which helped us find the issue causing that incompatibility with the two systems. In the end, we both slightly changed our logic to fit the pieces together, which made both systems work as intended again.

3.5 Fahim’s Part

Identification of classes

1. PlantPotScene.java
2. Plant2PotScene.java
3. FlowerScene.java
4. FlowerVaseScene.java
5. CactusPotScene.java
6. HiddenLetter.java
7. MailScene.java
8. OpenMailScene.java
9. TVGlitch.java
10. TvScene.java
11. Tvscene2.java

Class explanation: PlantPotScene.java , Plant2PotScene.java ,FlowerScene.java , FlowerVaseScene.java , CactusPotScene.java

These classes are responsible for constructing various 3D plant and vase objects in the scene. Each class provides a static method (such as createPlantPotBG() in PlantPotScene or createFlowerBG() in FlowerScene) that returns a BranchGroup containing all the elements required for that scene object. Individual components like pots, soil, leaves, stems, and flowers are each wrapped in their own TransformGroup, enabling precise control over their position, scale, and rotation within the scene. These TransformGroups are then assembled into a composite TransformGroup—often named something like plantPositionTG—to which overall transformations are applied. These transformations typically include translation to a specific

location in the main scene, scaling (such as 1.5 or 2.0 times the original size), and occasionally rotation, depending on how the object needs to be oriented.

All models are loaded using Java3D's ObjectFile loader and their corresponding textures are applied using TextureLoader. A helper method, usually named loadOBJWithTexture(), handles this by creating an Appearance object that includes texture bindings, material properties, and texture attributes like MODULATE mode to ensure the texture blends properly with lighting. To maintain modularity and reusability, each model is processed through a utility function (such as setAppearanceRecursively) that traverses the node tree and applies the appearance to all Shape3D nodes it encounters. This keeps the code clean and consistent across all classes.

The overall architecture is modular, each class is self-contained and responsible for loading its own models, setting up transformations, and assembling its part of the scene. This design makes integration into the larger environment simple, as the outer TransformGroup of each scene class allows it to be placed wherever needed in the main scene. Because all plant-related classes use the same utility methods and follow the same logic, they maintain a consistent appearance and behavior, ensuring that these natural elements blend seamlessly into the unified 3D space.

Class explanation: HiddenLetter.java, MailScene.java, OpenMailScene.java

The HiddenLetter system is designed to simulate a two-state mail interaction feature in the scene. It uses the createMailSystem() method to build and return a transformable mail object, which internally calls the create_Scene() method to construct the underlying BranchGroup. This scene comprises two main TransformGroups: mailRootTG and rotatingTG. mailRootTG handles the positioning and scaling of the entire mail system, placing it at a desired location in the room (e.g., at (30f, -15f, 90f)) and applying a uniform scaling factor like 0.5f. Nested inside it is rotatingTG, which is reserved for enabling future rotations of the mail independently of its position.

To manage the mail system's state, a Switch node called mailSwitch is used, allowing the application to toggle between two visual modes dynamically. Child 0 of this switch is the closed-envelope model created via MailScene.createMailBG(), while Child 1 is the open-envelope version generated by OpenMailScene.createOpenMailBG(), which includes an attached note. The method toggleMailState() manages the state-switching functionality: it checks which child is currently active, toggles it to the other, and plays a paper sound effect using the SoundUtilityJOAL instance for added immersion and auditory feedback.

Additional functionality includes the rotateMail(float degrees) method, which, although partially commented, is built to rotate the mail system using transformations on rotatingTG. For object

interaction, a PickTool is initialized to allow users to select and interact with the mail system through picking. Furthermore, lighting is configured through the createBasicLighting() method, which ensures that the mail object is well-illuminated and visually coherent with the rest of the scene.

The MailScene class is tasked with creating the visual representation of a closed envelope. It loads the mail.obj model and applies a texture named mail_texture.jpg. This model is then transformed by translating it to (30.0f, -17.0f, 90.0f), scaling it by a factor of 2.2, and rotating it first about the Y-axis (Math.PI / 6) and then about the X-axis (Math.PI / 2). These transformations are combined into a single TransformGroup which is then nested inside another outer group to allow further adjustments, such as repositioning or animation. A neutral background is added to keep the appearance uniform. When the HiddenLetter scene is assembled, MailScene.createMailBG() is added as Child 0 of the Switch node, making the closed-envelope view the default state.

On the other hand, the OpenMailScene class is responsible for creating the open-envelope view, which includes a note (paper) emerging from the envelope. It loads a different model (openmail.obj) and applies the texture openmail_texture.jpg. Similar transformations are applied to this envelope as well—translation, scaling, and a 90° Y-axis rotation to position it correctly. The note is generated using a method called createNoteShape(), which creates a textured quad that looks like a sheet of paper. This note is then positioned above the envelope with a 90° X-axis rotation, a 45° Y-axis rotation, and a scaling transformation to make it visually pop. The envelope and the note are grouped together and wrapped in a final TransformGroup that moves and scales the entire open mail object into position, usually at (30.0f, -10.0f, 90.0f) with a scale of 2.0. This assembled open-envelope view is added to the Switch node in HiddenLetter as Child 1 and becomes visible when toggled via toggleMailState().

Class Explanation: TvGlitch.java, TvScene.java, TvScene2.java

The TvGlitch.java, TvScene.java, and TvScene2.java classes together handle the creation and management of the interactive TV system in the 3D game environment. The main method responsible for building this system is createTVSystem() in TvGlitch.java, which constructs the visual structure using a BranchGroup assembled via a Switch node. This allows the program to dynamically toggle between the TV's normal and glitched appearances.

For positioning and controlling the TV, two TransformGroup nodes are used. The outer tvRootTG manages the TV's overall placement in the room by applying transformations such as translation (27.0f, 3.0f, -95.0f) and a scaling factor of 11.5f. Inside this, rotatingTG handles rotation-specific

functionality through the rotateTV() method. This structure ensures the TV can be oriented freely without disrupting its main position.

State management is handled via a Switch node named tvSwitch. This node toggles between two child nodes: Child 0, created using TvScene.createTvSceneBG(), represents the normal TV state, while Child 1, created using TvScene2.createTvSceneBG(), displays a glitched version that includes animated texture switching to simulate screen distortion. The method toggleTVState() controls which child is displayed. When toggled to the glitched state, a looping glitch sound effect is played using SoundUtilityJOAL, with the flag isGlitchPlaying ensuring the sound isn't redundantly triggered. When toggled back to the normal state, the glitch sound is stopped.

The rotateTV(float degrees) method provides further interactivity by enabling the TV to rotate based on user input. It reads the current transformation from rotatingTG, applies a rotation transformation about the Y-axis using Math.toRadians(degrees), and sets the updated transformation.

A PickTool is also initialized in the scene to enable object picking for mouse interaction, if required. Additionally, the scene is illuminated using basic DirectionalLight and AmbientLight elements, ensuring the TV appears well-lit and maintains visual consistency across states.

TvScene.java is responsible for rendering the standard TV view. It creates a TVFrame for the outer casing and a TVScreen for the display. These are combined into a TransformGroup and returned as a BranchGroup, which is used as Child 0 in the Switch node.

In contrast, TvScene2.java simulates a malfunctioning TV using a dynamic texture-switching mechanism. It loads two different textures and uses a Timer to alternate between them at a 500-millisecond interval. This simulates a visual glitch effect. The appearance is applied to the screen part of the TV using an Appearance object that supports texture updates. This scene is returned as Child 1 in the tvSwitch.

Integration within TvGlitch begins when createTVSystem() sets the initial state of the Switch node to Child 0. Users can toggle between normal and glitched states via toggleTVState(), causing the appropriate visual and audio feedback to occur.

The TV system is then integrated into the main environment through the BODMain class. This class acts as the central controller, combining all interactive elements into a unified virtual space. It calls methods like TVGlitch.createTVSystem() and places the resulting TransformGroup into the main BranchGroup. Along with other elements such as the mail system and plant models, the TV contributes to the immersive scene. The BODMain class also handles user input and navigation. It listens for keyboard and mouse events to control movement, camera rotation, and trigger object interactions like toggling the TV state. The user is restricted within certain movement

boundaries, and motion effects like "bouncing" simulate realistic exploration. The class sets up the rendering universe via SimpleUniverse and positions the camera using a predefined viewer setup, ensuring that all components are rendered and viewed correctly in the 3D space.

Techniques of Implementation in BODMain.java

BODMain acts as the main controller and scene integrator for the entire virtual environment. It builds the complete 3D scene using the create_Scene() method, which initializes the room (Room.createEmptyRoom()), adds components like PlantPotScene, CactusPotScene, HiddenLetter, and TVGlitch, and sets up lighting via Lights.setupSceneEffects(). Each object is added using its own TransformGroup, allowing individual scaling, positioning, and rotation. The assembled scene is compiled for performance.

In the constructor, Canvas3D and SimpleUniverse are set up for rendering. The viewer's position is defined using CommonsFK.define_Viewer(), and the compiled scene is attached to the universe. Since BODMain extends JPanel, it embeds the 3D canvas and is shown in a JFrame.

For user interaction, BODMain implements KeyListener, handling key events through keyPressed(). It supports camera movement, rotation, crouch toggling, and calls to interact with other systems like the mail or TV. This modular structure keeps the application responsive and organized.

Software Testing and Operation

For the **HiddenLetter and Mail System** (including MailScene and OpenMailScene), unit testing verified that createMailSystem() correctly generated the mail scene with proper translation (30f, -15f, 90f), scaling 0.5f, and rotation capability via rotatingTG. The toggleMailState() method was tested by toggling the mail multiple times and confirming the Switch node transitioned between the closed (MailScene) and open (OpenMailScene) states. It was also confirmed that the paper sound played only when switching to the open state.

Model loading tests validated that mail.obj and openmail.obj were properly loaded and textured. In MailScene, the envelope's positioning and rotations were accurate, and in OpenMailScene, the note created with createNoteShape() was correctly placed with appropriate transformations and scaling. Integration into BODMain ensured the mail system appeared correctly in the room, and that toggling was visually and audibly synchronized without lag.

For the **TV System** (TVGlitch, TvScene, and TvScene2), unit testing confirmed that createTVSystem() applied the intended translation (27.0f, 3.0f, -95.0f) and scale 11.5f, with rotatingTG enabling Y-axis rotation. The toggleTVState() method was tested to ensure it switched between the normal (TvScene) and glitched (TvScene2) states. The glitch sound was confirmed to loop correctly only in the glitched state and stopped when toggled back.

TvScene2 was tested for dynamic texture switching, confirming two textures alternated every 500ms using a Timer. Integration into BODMain verified proper placement in the scene and smooth interaction. All TV states transitioned correctly, and their visual/audio effects functioned without interrupting other elements in the environment.

4. Discussions and Recommendations

4.1 Discussions About the Team Projects

The development of this project required extensive collaboration, effective planning, and technical problem-solving. Utilizing the Scrum methodology, our team organized tasks into iterative sprints, ensuring continuous progress and adaptability to evolving challenges.

Teamwork Experience

Throughout the project, teamwork played a crucial role in successfully integrating 3D modeling, Java programming, and interactive components. Regular team meetings facilitated open discussions, enabling us to address obstacles and allocate resources effectively. Each member contributed based on their expertise, ensuring a balanced workload.

Challenges Encountered

Despite our structured approach, we faced several challenges, including:

- Technical Complexity: Implementing realistic 3D models, animations, and physics simulations in Java 3D posed significant difficulties.
- Performance Optimization: Rendering high-quality 3D graphics while maintaining smooth performance required extensive debugging and optimizations.
- Time Constraints: Balancing academic responsibilities with project deadlines was challenging, particularly during final implementation and testing phases.
- Version Control and Integration Issues: Coordinating work among multiple contributors occasionally led to merge conflicts and compatibility problems.

Solutions Implemented

To address these challenges, we adopted the following strategies:

- Regular team meetings and progress updates to ensure alignment and resolve issues promptly.
- Use of Git for version control, ensuring smooth collaboration and preventing code conflicts.
- Division of tasks based on team members' strengths, allowing for more efficient development.

- Early and frequent testing, identifying and fixing issues at each stage rather than at the end.

By implementing these strategies, we were able to enhance productivity, improve problem-solving efficiency, and maintain project momentum.

4.2 Recommendations for Better Practices

Improvements for Future Projects

Based on our experience, we recommend the following best practices to enhance future software development projects:

- More Effective Time Management: Allocating sufficient buffer time for debugging, optimization, and user testing, rather than focusing solely on feature implementation. This ensures a stable and polished final product.
- Advanced Testing Strategies: Implementing automated unit testing and integration testing to improve efficiency, reduce human error, and detect issues early in development.
- Improved Documentation Standards: Maintaining detailed class descriptions, inline comments, UML diagrams, and a structured project report to facilitate future maintenance, scalability, and troubleshooting.
- More Granular Sprint Planning: Breaking tasks into smaller, well-defined sub-tasks to improve tracking, accountability, and overall productivity. This ensures smoother task delegation and reduces bottlenecks.

Key Lessons Learned

This project provided valuable insights into software development, team collaboration, and problem-solving under real-world constraints. Key takeaways include:

- Clear communication and collaboration are essential for managing complex development projects efficiently. Effective use of Scrum methodologies helped us maintain progress and resolve issues quickly.
- Early identification of technical challenges—such as rendering performance issues and animation synchronization—enabled us to address problems proactively rather than reactively.

- Comprehensive documentation and structured coding practices significantly improved maintainability and debugging, ensuring long-term usability and extensibility of the software.

5. Conclusion

- **Project outcomes**

The final outcome of our project is a fully interactive and immersive 3D game experience that brings together technical depth, creativity, and narrative design. Set in a mysterious, dimly lit room, the game invites the player to explore, investigate, and uncover clues to piece together the story behind a suspicious death. The environment is populated with a wide range of interactive objects—such as a blood-stained knife, a broken clock, scattered furniture, and a vintage record player—each carefully placed to contribute to the overall atmosphere and mystery. As players interact with these objects, they gradually reveal elements of the storyline, guided by visual and audio cues that help build suspense and immersion.

From a technical standpoint, the project met all of our core development goals. We implemented realistic 3D object rendering, smooth animations, user-controlled first-person navigation, and collision detection to guide movement within the scene. Interactive features such as object-based dialogue triggers, sound effects, and animated components like the spinning record and ticking clock enriched the gameplay experience. These combined features allowed us to create a game that feels alive and responsive, where every element contributes to a deeper sense of presence and engagement.

Just as importantly, the project served as a valuable opportunity for team growth and practical learning. Working within an agile framework, we applied Scrum principles to organize and prioritize our tasks, track progress, and adapt to changing requirements. We developed our collaboration and planning skills by holding regular team discussions and using Microsoft Azure DevOps to manage our backlog and sprints. Through these efforts, we not only delivered a successful game project, but also gained hands-on experience with real-world development workflows and the challenges of building software in a team setting.

- **Key achievements**

One of the primary achievements of our project was the successful development of a fully functional and immersive 3D game environment using Java 3D and JOGL. We designed and implemented a wide variety of detailed and textured 3D models, each with its own behaviors, interactions, and transformations. The game includes a first-person camera system, collision detection, and dynamic object placement. We also implemented animations, such as a continuously spinning record player and a working clock mechanism, which added a sense of realism and interactivity to the experience.

We further enhanced the player experience by integrating sound effects tied to specific events and background ambiance, adding depth and atmosphere to the scene. Additionally, we implemented UI elements like overlay dialogue boxes and temporary 2D image displays, which allowed us to communicate in-game clues, feedback, and narrative elements to the player. These features were crucial in creating a cohesive and engaging user experience that went beyond static 3D visuals.

Beyond technical execution, a major success was our development as a collaborative team. This project provided us with hands-on experience using Scrum methodology to plan and track our progress in sprints, and we organized our workflow using Microsoft Azure DevOps. This structure helped us improve in key areas such as communication, task delegation, time management, and problem-solving. Learning how to navigate real-world collaboration tools and agile practices was a valuable experience that not only helped us complete the project efficiently but also prepared us for future team-based software development.

6. Signatures for Contribution/Participation Tables

In submitting this team project and report, we confirm that the contribution and the participation table both represent accurately the amount of work we did, and the effort we made for the project.

Signatures: 

7. List of Backlogs

- Sprint 1 : Creating Objects
    - Creates specific models needed for the scene , and setting up github and other essential stuffs

	1	GitHub Setup	Done		Marco Lee-Shi
		Define Project's Structure	Done		Marco Lee-Shi
	2	Set Up the 3DRoom	Done		Lynn Hajj Hassan
		Create an empty 3Droom	Done		Lynn Hajj Hassan
		User Navigation	Done		Lynn Hajj Hassan
+	3	Noor's Objects	Done	⋮	Noor Haddad
		Corpse	Done		Noor Haddad
		Sofa	Done		Noor Haddad
		Table(s)	Done		Noor Haddad
		Double Bass	Done	⋮	Noor Haddad
		Make Separable Classes for the Objects	Done		Noor Haddad

	4	Hanan's Objects	Done		Hanan Senah
		Drugs to plant	Done		Hanan Senah
		Chair	Done		Hanan Senah
		Clock	Done		Hanan Senah
		Mini Side Table	Done		Hanan Senah
		Windows and the view Outside	Done		Hanan Senah
		Make Separable Classes for the Objects	Done		Hanan Senah
		frame and team picture	Done		Hanan Senah
+	5	Lynn's Objects	Done		Lynn Hajj Hassan
		Table Lamp	Done		Lynn Hajj Hassan
		Ceiling Lamp	Done		Lynn Hajj Hassan
		Mirror	Done		Lynn Hajj Hassan
	6	Fahim's Objects	Done		Fahim Kabir
		vase/pots/flowers	Done		Fahim Kabir
		Belladonna flower	Done		Fahim Kabir
		TV	Done		Fahim Kabir
		Make Separable Classes for the Objects	Done		Fahim Kabir
		The paper note	Done		Fahim Kabir
		The Paper Note	Done		Fahim Kabir
+	7	Marco's Objects	Done		Marco Lee-Shi
		trash can	Done		Marco Lee-Shi
		pillow(s)	Done		Marco Lee-Shi
		Improve User's Navigation	Done		Marco Lee-Shi
		The blood and the knife	Done		Marco Lee-Shi
		Music Player	Done		Marco Lee-Shi

- Sprint 2: Placing models inside the room
  Placing models created inside the empty room

	1	Placing Lynn's Objects	Done	Lynn Hajj Hassan
		placing the lamp ceiling	Done	Lynn Hajj Hassan
		placing the table's lamp	Done	Lynn Hajj Hassan
		Placing the mirror	Done	Lynn Hajj Hassan
	2	Placing Marco's Objects	Done	Marco Lee-Shi
		placing the chocolate box	Done	Marco Lee-Shi
		placing the garbage	Done	Marco Lee-Shi
		placing the pillow	Done	Marco Lee-Shi
+	3	Placing Noor's Objects	Done	Noor Haddad
		placing the corpse	Done	Noor Haddad
		Placing the sofa	Done	Noor Haddad
		Placing the violin	Done	Noor Haddad
		Placing the table	Done	Noor Haddad
		Placing the pillow	Done	Noor Haddad
		Placing the table	Done	Noor Haddad
		Placing the pillow	Done	Noor Haddad
		Placing the carpet	Done	Noor Haddad
	4	Placing Hanan's Objects	Done	Hanan Senah
		placing the clock	Done	Hanan Senah
		placing the pills	Done	Hanan Senah
		placing the chair	Done	Hanan Senah
		placing the small table	Done	Hanan Senah
		Placing the Window	Done	Hanan Senah
+	5	Placing Fahim's Objects	Done	Fahim Kabir
		placing the tv	Done	Fahim Kabir
		placing the vases	Done	Fahim Kabir
		placing the flower	Done	Fahim Kabir
		placing other objects made by him	Done	Fahim Kabir

- Sprint 3: Animate models, and integrate user's interaction with the environment.

1	Marco's Part	Done		Marco Lee-Shi
	2d Image Displaying Class	Done		Marco Lee-Shi
	TriggerEvent Part	Done		Marco Lee-Shi
	Recording Audios and Finding Sounds	Done		Marco Lee-Shi
2	Hanan's Part	Done		Hanan Senah
	Animating the clock	Done		Hanan Senah
	Helping animating the Tv	Done		Hanan Senah
	Audio Managing Class	Done		Hanan Senah
	TriggerEvents Part	Done		Hanan Senah
	Recording And Finding Sounds	Done		Hanan Senah
3	Fahim's Part	Done		Fahim Kabir
	Paper Note's Animation	Done		Fahim Kabir
	Tv's Animation	Done		Fahim Kabir
	TriggerEvent Part	Done		Fahim Kabir
4	Noor's Part	Done		Noor Haddad
	CameraEffects Class	Done		Noor Haddad
	Objects Collision	Done		Noor Haddad
	TriggerEvents Part	Done		Noor Haddad
	Additional Useful Features	Done		Noor Haddad
	Collecting Some Audios	Done		Noor Haddad
5	Lynn's Part	Done		Lynn Hajj Hassan
	Plan Animation Flow and Templates	Done		Lynn Hajj Hassan
	InnerThought Class	Done		Lynn Hajj Hassan
	GameState Class	Done		Lynn Hajj Hassan
	Clicks Handling	Done		Lynn Hajj Hassan
	Lights Animation	Done		Lynn Hajj Hassan
	TriggerEvents Part	Done		Lynn Hajj Hassan

- Sprint 4 : Make the final video

1	Making the Submission Video	Done		Marco Lee-Shi
	Editing the video	Done		Marco Lee-Shi
	Drawing the 2d image	Done		Lynn Hajj Hassan